

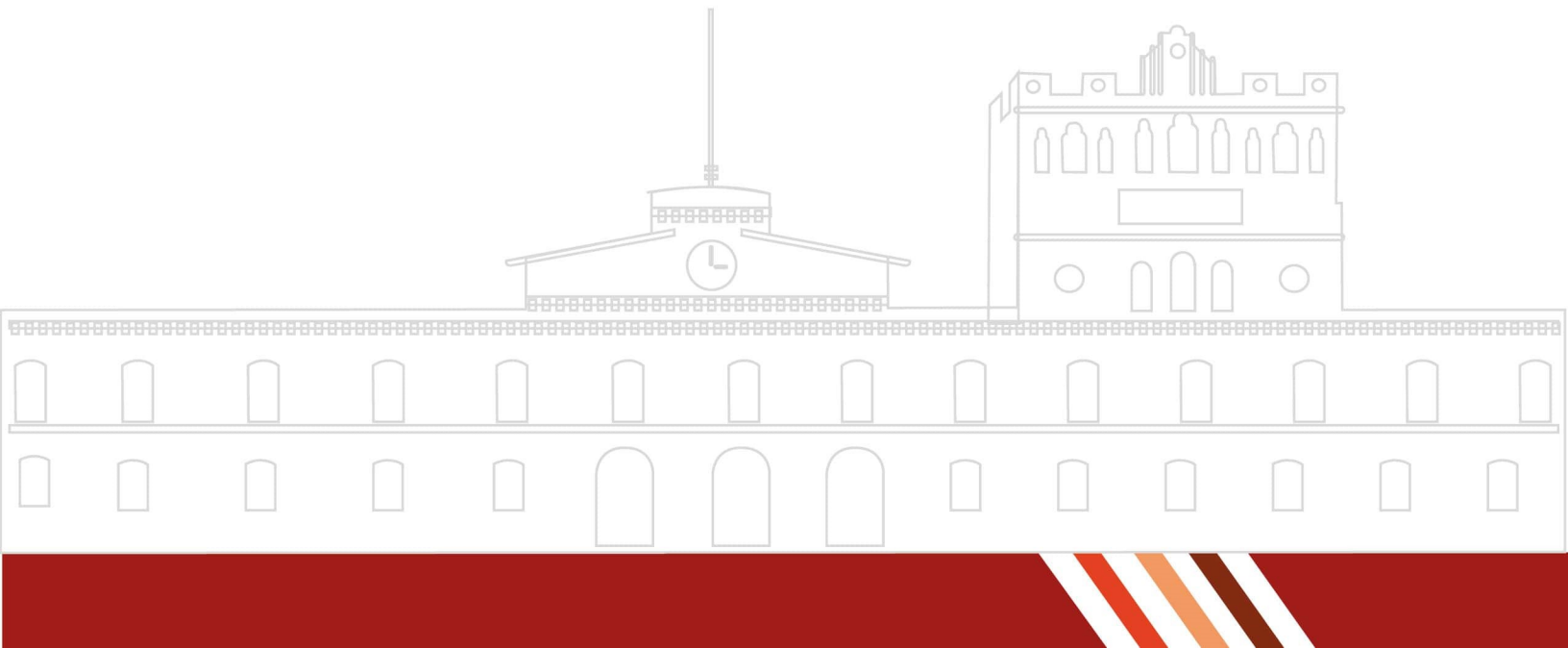
INVESTIGACIÓN

¿QUÉ ES UN AUTÓMATA FINITO?

ALUMNO:

SAMUEL ORTIZ SALINAS

Dr. Eduardo Cornejo Velázquez



¿Qué es un autómata finito?

Un autómata finito es un modelo matemático de una máquina abstracta que recibe una cadena de símbolos, la procesa de izquierda a derecha y determina si pertenece o no a un lenguaje. Se denomina finito porque dispone de una cantidad limitada de estados. Durante el procesamiento, la máquina se encuentra en uno de esos estados y cambia de uno a otro según el símbolo leído y las reglas establecidas en su función de transición.

El autómata comienza en un estado inicial. Después lee cada símbolo de la cadena y realiza la transición correspondiente. Cuando termina la entrada, la cadena es aceptada si el autómata se encuentra en un estado final o de aceptación; en caso contrario, es rechazada. Su propósito es describir formalmente un procedimiento de reconocimiento, sin necesidad de representar una computadora física.

Los autómatas finitos están relacionados con los lenguajes regulares. Un lenguaje regular es un conjunto de cadenas que puede ser reconocido por algún autómata finito. Las expresiones regulares describen la misma clase de lenguajes.

Conceptos previos

Alfabeto

Un alfabeto es un conjunto finito y no vacío de símbolos y normalmente se representa mediante Σ . Por ejemplo:

$$\Sigma_1 = \{0, 1\}, \quad \Sigma_2 = \{a, b, c\}.$$

Cadena

Una cadena o palabra es una secuencia finita de símbolos de un alfabeto. Si $\Sigma = \{0, 1\}$, entonces 0, 101 y 00110 son cadenas válidas. La longitud de una cadena w se representa como $|w|$. La cadena vacía no contiene símbolos, se representa mediante ε y cumple $|\varepsilon| = 0$. El conjunto de todas las cadenas que pueden formarse sobre Σ se representa con Σ^* .

Lenguaje formal

Un lenguaje formal L es cualquier subconjunto de Σ^* . Por ejemplo:

$$L = \{w \in \{0, 1\}^* \mid w \text{ termina en } 1\}.$$

Este lenguaje contiene 1, 01, 101 y 1101, pero no contiene 0, 10 ni 110.

Definición formal

Un autómata finito determinista se define mediante la quintupla

$$M = (Q, \Sigma, \delta, q_0, F),$$

donde:

- Q es un conjunto finito y no vacío de estados;
- Σ es el alfabeto finito de símbolos de entrada;
- δ es la función de transición;
- q_0 es el estado inicial, con $q_0 \in Q$;
- F es el conjunto de estados finales, con $F \subseteq Q$.

Los estados representan las situaciones internas posibles. El alfabeto determina los símbolos admitidos. La función de transición establece cómo debe reaccionar la máquina. El estado inicial señala el comienzo y F identifica los resultados de aceptación.

Función de transición

En un autómata finito determinista, la función de transición es

$$\delta : Q \times \Sigma \longrightarrow Q.$$

La expresión $\delta(q, a) = p$ indica que, si el autómata está en q y recibe a , cambia a p . Para procesar cadenas completas se utiliza la función extendida

$$\delta^* : Q \times \Sigma^* \longrightarrow Q,$$

definida por

$$\delta^*(q, \varepsilon) = q, \quad \delta^*(q, xa) = \delta(\delta^*(q, x), a).$$

La primera regla indica que leer la cadena vacía no cambia el estado. La segunda permite calcular el estado alcanzado después de leer una cadena símbolo por símbolo.

Aceptación de cadenas

Una cadena w es aceptada cuando el procesamiento desde el estado inicial termina en un estado perteneciente a F :

$$\delta^*(q_0, w) \in F.$$

El lenguaje reconocido por M se define como

$$L(M) = \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\}.$$

Si el estado alcanzado no pertenece a F , la cadena es rechazada. La aceptación depende del estado en que termina la lectura, no de haber pasado antes por un estado final.

Representación

Diagrama de transición

Un autómata puede representarse mediante un grafo dirigido. Cada nodo corresponde a un estado y cada flecha representa una transición. Su etiqueta indica el símbolo que produce el cambio. El estado inicial se distingue mediante una flecha que llega desde fuera del diagrama y los estados finales suelen representarse con un círculo doble.

Tabla de transición

La función δ también puede organizarse en una tabla. Las filas representan estados, las columnas representan símbolos y cada celda contiene el estado siguiente.

Cuadro 1: Autómata que reconoce cadenas binarias terminadas en 1.

Estado	Entrada 0	Entrada 1
$\rightarrow q_0$	q_0	q_1
$*q_1$	q_0	q_1

Ejemplo de funcionamiento

Considérese el autómata

$$M = (\{q_0, q_1\}, \{0, 1\}, \delta, q_0, \{q_1\}).$$

Para procesar 101, se realizan los movimientos

$$q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_0 \xrightarrow{1} q_1.$$

La cadena es aceptada porque termina en q_1 . Para 110, el recorrido es

$$q_0 \xrightarrow{1} q_1 \xrightarrow{1} q_1 \xrightarrow{0} q_0.$$

Esta cadena es rechazada porque termina en q_0 .

Tipos de autómatas finitos

Autómata finito determinista

Un autómata finito determinista (AFD) tiene una sola transición para cada combinación de estado y símbolo. Para una misma entrada existe un único recorrido posible. Cuando falta alguna transición puede agregarse un estado de error o sumidero.

Autómata finito no determinista

En un autómata finito no determinista (AFND), una combinación de estado y símbolo puede conducir a varios estados:

$$\delta : Q \times \Sigma \longrightarrow \mathcal{P}(Q),$$

donde $\mathcal{P}(Q)$ es el conjunto potencia de Q . Una cadena se acepta si al menos uno de los recorridos posibles termina en un estado final. Algunos AFND permiten transiciones ε , que cambian de estado sin consumir un símbolo. Los AFD, los AFND y los AFND con transiciones ε reconocen exactamente los lenguajes regulares.

Equivalencia y conversión

Para todo AFND existe un AFD que acepta el mismo lenguaje. La conversión se realiza mediante la construcción de subconjuntos: cada estado del AFD representa un conjunto de estados posibles del AFND. Si el AFND tiene n estados, el AFD equivalente puede requerir hasta 2^n estados.

Dos autómatas son equivalentes cuando reconocen el mismo lenguaje:

$$L(M_1) = L(M_2).$$

No necesitan tener la misma cantidad de estados ni las mismas transiciones.

Expresiones regulares

Los autómatas finitos y las expresiones regulares tienen el mismo poder de representación. Para toda expresión regular puede construirse un autómata que reconozca su lenguaje y para todo autómata finito puede obtenerse una expresión regular equivalente.

El lenguaje de cadenas binarias terminadas en 1 puede describirse como

$$(0|1)^*1.$$

Esta equivalencia se utiliza en analizadores léxicos, búsquedas de patrones y validadores de entradas.

Minimización

La minimización consiste en obtener un AFD equivalente con la menor cantidad posible de estados. Primero pueden eliminarse los estados inaccesibles, a los que no existe un camino desde el estado inicial. Después se identifican estados equivalentes. Dos estados son

equivalentes cuando ninguna cadena permite distinguirlos con respecto a la aceptación. Al agruparlos se reduce el tamaño sin modificar el lenguaje reconocido.

Aplicaciones

Los autómatas finitos se emplean en:

- análisis léxico de compiladores;
- reconocimiento de identificadores, números, operadores y palabras reservadas;
- búsqueda de patrones en texto;
- validación de correos, códigos, fechas y formatos;
- diseño de protocolos de comunicación;
- modelado de interfaces, menús y videojuegos;
- control de semáforos, elevadores y máquinas expendedoras;
- diseño de circuitos secuenciales;
- detección de secuencias en conjuntos de datos.

En un analizador léxico, cada clase de token puede expresarse como un lenguaje regular. El autómata lee los caracteres del código fuente y cambia de estado hasta reconocer un identificador, número, operador u otro símbolo.

Ventajas y limitaciones

Entre sus ventajas se encuentran su precisión matemática, su representación clara y su procesamiento eficiente. Para una cadena de longitud n , un AFD realiza el reconocimiento en tiempo lineal porque consume cada símbolo una vez.

Su principal limitación es la memoria: solo conserva información mediante el estado actual y tiene una cantidad finita de estados. Por ello no puede reconocer lenguajes que exigen recordar cantidades arbitrarias. Por ejemplo:

$$L = \{a^n b^n \mid n \geq 0\}.$$

Para reconocer este lenguaje tendría que recordar cuántas letras a aparecieron y compararlas con las letras b . En esos casos se necesita un modelo con memoria adicional, como un autómata de pila.

Referencias

- [1] Brookshear, J. G. (1999). *Teoría de la computación: Lenguajes formales, autómatas y complejidad*. Pearson Educación.
- [2] Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2002). *Introducción a la teoría de autómatas, lenguajes y computación* (M. Alfonseca, Á. M. Ortigosa, E. Pulido y P. Rodríguez, Trads.; 2.^a ed.). Pearson Educación.
- [3] Kelley, D. (1995). *Teoría de autómatas y lenguajes formales* (M. L. Díez Platas, Trad.). Prentice Hall.